EMERGEX

Campus-Linked Student Accommodation Search and Viewing-Request System

Phase 5 — Implementation

IT Project 700 · Semester 2, 2026

Richfield Graduate Institute of Technology · Faculty of Information Technology

Group 55 · Supervisor: Thabiso (ThabisoA@richfield.ac.za)

Due 9 November 2026

Group members

Student no.	Name	Surname	Email	Contact no.
402308936	Tristan	Moses	402308936@my.richfield.ac.za	082 801 5082
402411668	Reenesh	Ramawthar	402411668@my.richfield.ac.za	084 604 1521
402411227	Joyce	Kamalu	402411227@my.richfield.ac.za	067 107 5365
402412304	Novithan	Naidoo	402412304@my.richfield.ac.za	062 082 9186
402413409	Thingo	Shezi	402413409@my.richfield.ac.za	071 429 9251
402111846	Lungelo	Ngwane	402111846@my.richfield.ac.za	064 253 4563

Table of Contents

5.1 Introduction....4
5.1.1 Purpose of this phase....4
5.1.2 What was delivered....4
5.1.3 Technology....6
5.1.4 Summary of outcomes....6
5.2 Coding....8
5.2.1 Architecture as built....8
5.2.2 Coding conventions....8
5.2.3 Representative modules....10
5.2.4 Security as implemented....15
5.2.5 Known limitations in the delivered code....15
5.3 Testing....17
5.3.1 Objectives....17
5.3.2 Scope....17
5.3.3 Test strategy — the six levels....18
5.3.4 Test environment....20
5.3.5 Tools, and why they changed....21
5.3.6 Framework architecture....22
5.3.7 Test design principles....24
No selector appears in a test.....24
5.3.8 Entry and exit criteria....24
5.3.9 Executing the suite....25
5.4 System Testing....26
5.4.1 Test case design....26
5.4.2 Case status and execution result....26
5.4.3 The test runs of record....26
5.4.4 Results....28
5.4.5 Traceability....30
5.4.6 Defects identified....33
5.4.7 Corrections, and verifying them (TR-002)....39
5.4.8 Evaluation of the testing results....41

5.4.9 Limitations 43
5.4.10 Recommendations 43
5.5 Installation 44
5.5.1 Requirements 44
5.5.2 Installing the application 44
5. Create the configuration: 44
6. Load the reference data: 44
5.5.3 Verifying the installation 46
5.5.4 Installing and running the test suite 46
5.5.5 Troubleshooting 48
5.5.6 Backup and recovery 50
Conclusion 51
Appendix A — Test case specifications 52
Appendix B — Reproducing the test run 58
B.1 Prerequisites 58
B.2 Procedure 58
B.3 Expected outcome 58
B.4 Repository contents 59
References 59

5.1 Introduction

5.1.1 Purpose of this phase

The design created in Phase 4 is transformed into a functioning, tested system in the Implementation phase. Phase 3 identified what EMERGEX must do, and Phase 4 identified how EMERGEX would be built. This phase describes what EMERGEX actually built, what it did to prove it worked, what that proving found, and how it was installed and run.

There are five sections to the phase. It says what has been delivered. Code of the application section 5.2 Testing is covered in Sections 5.3 and 5.4, starting with the strategy, then the test cases (execution against the dated run) and finally the defects this execution brought out. The process for installation is in section 5.5

The most important outcome is found in 5.4.6. Seven flaws were identified, all of which the group's own Phase 3 analysis had predicted before any test had been written. The focus of this phase is the chain that consists of a weakness that analysis identified, a correction that design specified and a test failure that demonstrates the correction is still needed.

5.1.2 What was delivered

EMERGEX serves Richfield students with accommodation request and viewing functionality. Richfield does not own any student residences and informally, students are referred to off-campus housing. Through classified sites, social media groups, and word of mouth, students must assemble a search for housing. Phase 3 of investigations found eight weaknesses in the processes, all of which link to the same cause. There is no single source of structured programmed accommodation information verified at the campus level.

Table 5.1.1 Delivered functionality

Req.	Capability	Where it is implemented
FR1	Every campus shown with city, province, confirmed status and bed count	index.php
FR2	Search by campus, maximum rent, room type and amenities	search.php, repository.php
FR3	Rent, room type, inspected status, amenities and travel times per listing	listing.php, listing-card.php
FR4	Listings and their campus plotted on an interactive map	app.jswith Leaflet and OpenStreetMap
FR5	Shortlist of up to four rooms, kept across pages and reloads	app.js, browser storage
FR6	Viewing request captured, referenced and stored	apply.php, repository.php

FR7	All input validated on the server	apply.php
FR8	Move-in cost estimated for a chosen lease and payment plan	functions.php, mirrored in app.js
FR9	Form protected against cross-site request forgery	apply.php
FR10	Site keeps working with no database connected	The seam in functions.php

Alongside the application, an automated test suite of 229 tests was built to verify it. That suite is a deliverable of this phase in its own right and is described in sections 5.3 and 5.4.

5.1.3 Technology

Table 5.1.2 — Implementation technologies

Layer	Technology	Reason
Web server	Apache (XAMPP)	Prescribed for the module; matches the assessment environment
Application	PHP 8, no framework	A framework would hide the mechanisms being assessed — routing, sessions, escaping and prepared statements are all visible in the source
Database	MySQL 8 / MariaDB, InnoDB	Foreign keys and transactions are required: a request and its rooms must both be written or neither
Database access	PDO, native prepared statements	ATTR_EMULATE_PREPARES => falsesends parameters separately from the statement, which is what actually prevents injection
Front end	Hand-written HTML, CSS, JavaScript	No build step and no dependencies to install or break (NFR1)
Test automation	Python 3.13 · pytest 8.3 · Playwright 0.6	See section 5.3.5
Test case management	BeyondTheBugs, built by the a single member of the group which serves as a project website and testing tool.	See section 5.3.5

5.1.4 Summary of outcomes

Table 5.1.3 Phase 5 at a glance

Measure	Result
Test cases designed	60, every one traced to a requirement
Automated tests implemented	149 test functions, 230 executions, across ten suite files

Test runs of record	TR-001 and TR-002, both 1 November 2026
Tests executed, each run	224
TR-001 — before correction	217 passed, 7 failed · pass rate 96.9%
TR-002 — after correction	**224 passed, 0 failed · pass rate 100%**
Defects identified	7
Defects corrected and verified	7 of 7
Defects predicted by the Phase 3 analysis	7 of 7
Unexplained failures	0

5.2 Coding

5.2.1 Architecture as built

The application follows the four-layer design specified in Phase 4 4.3.1, with one PHP file per user-facing address. Two rules make the layering real rather than aspirational, and both can be checked by inspection: **no page contains SQL**, and **only one file opens a database connection**.

```
Richfield accommodation interface/
├── index.php          Landing page — campuses, statistics, cost estimator, FAQ
├── search.php         Search, filters and map
├── listing.php        Room detail
├── apply.php          Viewing-request form, validation and save
│
├── includes/
│   ├── db.php             [DB-CONNECTOR] the only `new PDO`
│   ├── repository.php     [DB-QUERIES] every SQL statement
│   ├── functions.php      Distance, money, formatting, the data-source seam
│   ├── data.php           Sample data: fallback source and seed source
│   ├── config.php         Credentials — git-ignored
│   ├── header.php / footer.php / listing-card.php
│
├── database/
│   ├── schema.mysql.sql   Nine tables for MySQL / MariaDB
│   ├── schema.sqlite.sql  The same for SQLite
│   └── seed.php           Creates the tables and loads reference data
│
└── assets/
    ├── css/style.css      The entire design system
    └── js/app.js          Shortlist, calculator, map
```

Nineteen files in total. The convention markers in square brackets are searchable: a single search for [DB- lists every point at which the application touches data, which is the review path a maintainer or an assessor would follow.

5.2.2 Coding conventions

- **Every database query is a prepared statement.** There is no concatenation of user input into SQL anywhere in the codebase.
- **Every variable echoed into HTML passes through e().** Unescaped output is how stored content becomes executable script.
- **Every state-changing POST verifies a CSRF token** before doing anything else.
- **Money is stored and calculated in whole rand.** The estimator rounds at every step; storing cents would introduce drift between the stored figure and the displayed one.
- **Displayed statistics are derived, never hardcoded.** Total beds, inspected count and median walking time are computed on each request.
- **Optional dependencies are wrapped.** mbstringis not present in a stock PHP build, so str_len()falls back to

strlen() rather than making the form fatal.

5.2.3 Representative modules

Six modules are described below. Between them they cover the connection, the queries, the business logic, the one write path, the main read path and the data layer.

Module 1 — includes/db.php(the connector)

The only file in the project that creates a database connection. It opens one PDO instance lazily, reuses it, and supports MySQL and SQLite behind the same interface.

```
$pdo = new PDO($dsn, $cfg['username'], $cfg['password'], [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,  // fail loudly
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,                   // real prepares
]);
```

ATTR_EMULATE_PREPARES => false is the important option. With emulation on, PDO builds the final SQL string itself and only *appears* parameterised. With it off, parameters travel to the database separately from the statement, so user input can never be parsed as SQL. Test cases TC-45, TC-46 and the injection tests in 5.4 verify the result.

The connector also degrades rather than failing: with no configuration file, or with the server down, db() returns null and the site runs on sample data. A strictflag inverts this once the database is the source of truth.

Module 2 — includes/repository.php(every query)

All SQL in the project lives here. The search query is the most interesting, because its *shape* varies with the filters while every *value* remains bound.

```
$sql    = repo_listing_select() . ' WHERE campus_id = ? AND is_active = 1 AND rent <= ?';
$params = [$campus_id, $max_rent];

if ($types) {
    $sql .= ' AND room_type IN (' . implode(',', array_fill(0, count($types), '?')) . ')';
    $params = array_merge($params, array_values($types));
}

if ($wanted) {
    $in   = implode(',', array_fill(0, count($wanted), '?'));
    $need = (int) count($wanted);          // interpolated, see below

    $sql .= " AND slug IN (
                SELECT listing_slug FROM listing_amenities
                 WHERE amenity_id IN ($in)
                 GROUP BY listing_slug
                HAVING COUNT(DISTINCT amenity_id) = $need
              )";
    $params = array_merge($params, array_values($wanted));
}
```

We need to draw out two choices. The counts of the selected values are what build the placeholder lists; not the selected values themselves, which allows for a variable number of filters to be used without needing to concatenate input.

The reason for the interpolated configuration of the HAVING count is informative. When emulation is turned off, a bound value shows up as a string. SQLite effectively compares COUNT(...), an integer, with a string '2', and there’s never a match.

when MySQL coerces and operates. One of the two supported engines would only make a defect apparent. Interpolating a value here is safe because that value is count() of a PHP array, casted to an integer which can never contain user input. The amenities in test case TC-12 are AND, not OR.

Module 3 — includes/functions.php(logic and the seam)

Holds the business calculations and the single branch that satisfies FR10.

```
function all_listings(): array
{
    static $cache = null;
    if ($cache === null) {
        $cache = using_database() ? repo_listings() : LISTINGS;
        if (!$cache) {
            $cache = LISTINGS;   // connected but the table is empty
        }
    }
    return $cache;
}
```

Five lookup functions follow this pattern. Since all pages read data through these five functions and nothing else, all the lookup functions yield the same result. Since both the routes return an array of the same shape, the whole presentation layer is unaware of which source answered. That one design choice is why “works with no database” is a two-line branch and not parallel code.

The same file holds calculate_payment(), which implements BR-13. The rule that carries the most meaning is in the comment:

```
// Discounts apply to rent only, never to the deposit or admin fee - a
// deposit is refundable and discounting it would misrepresent what is held.
$discount   = (int) round($gross * $rate);
$rent_total = (int) round($gross - $discount);
$deposit    = (int) round($rent * $deposit_months);
$total      = $rent_total + $deposit + $admin;
```

Test case TC-26 verifies precisely this by reading the deposit under all three payment plans and requiring the three to be equal.

Module 4 — apply.php(the only write path)

The only area where the system processes input given by a person – and as such, the most difficult test surface in the application. The order is important: validation of the token occurs before everything else, and opening of the transaction does not happen until every rule has passed.

```
if (!hash_equals($_SESSION['csrf'] ?? '', (string) ($_POST['csrf'] ?? ''))) {
    $errors['form'] = 'Your session expired. Please try again.';
}

if (str_len($values['fullName']) < 2)                    { $errors['fullName'] = 'Enter your full
name'; }
if (!filter_var($values['email'], FILTER_VALIDATE_EMAIL)) { $errors['email']    = 'Enter a valid email
address'; }

// South African mobile: 0 or +27, then a 6/7/8 prefix, then 8 digits.
if (!preg_match('/^(?:\+27|0)[6-8][0-9]{8}$/',
                preg_replace('/\s+/', '', $values['phone']))) {
    $errors['phone'] = 'Enter a South African mobile number';
}

if ($values['date'] < date('Y-m-d'))      { $errors['date']  = 'Choose a date that is not in the past';
}
if (str_len($values['notes']) > 500)      { $errors['notes'] = 'Keep it under 500 characters'; }
```

Errors are *collected* rather than thrown, so every failing field is reported together and the student's entries are preserved on redisplay. Test cases verify both behaviours.

Three checks are absent from that sequence, and their absence is the substance of three of the seven defects in 5.4.6: there is no maximum length on the name, no check that at least one room was selected, and no check that the selected rooms belong to the selected campus.

Once validation passes, the write is atomic:

```
return db_transaction(function (PDO $pdo) use ($data, $listing_slugs) {
    $stmt = $pdo->prepare('INSERT INTO viewing_requests (...) VALUES (?,?,?,?,?,?,?,?,?,?)');
    $stmt->execute([...]);
    $id = (int) $pdo->lastInsertId();

    $room = $pdo->prepare('INSERT INTO viewing_request_rooms (request_id, listing_slug) VALUES (?,
?)');
    foreach ($listing_slugs as $slug) {
        $room->execute([$id, $slug]);
    }
    return $id;
});
```

Both inserts occur inside one transaction, so a request can never be stored without the rooms it names (BR-11). Any exception rolls the whole thing back and the student is told plainly, with the underlying error logged rather than displayed.

Module 5 — search.php(filter validation)

Filter state lives in the URL, so any filtered view is a shareable link. That also means every filter value arrives from the client and must be treated as untrusted.

```
$campus_id = isset($_GET['campus']) && get_campus($_GET['campus']) ? $_GET['campus'] : DEFAULT_CAMPUS;
$max_rent  = isset($_GET['max'])  ? max(2000, min(9000, (int) $_GET['max'])) : 9000;
$types     = isset($_GET['type'])    ? array_intersect((array) $_GET['type'],
array_keys(all_room_types())) : [];
$wanted    = isset($_GET['amenity']) ? array_intersect((array) $_GET['amenity'],
array_keys(all_amenities()))  : [];
```

In different lines, we see different versions of validation chosen to match the value. The campus must resolve to a real campus or else the default is used. The rent is clamped into range rather than just rejected so that an edited link still works. The room-type and amenity lists are intersected against known keys. So an unknown value is simply dropped. The clamping at four boundaries is verified via test case TC-10.

This is the validation the viewing-request form does not do for its campus and move-in month — the same codebase, two different standards, which is how DEF-04 and DEF-05 came about.

Module 6 — database/schema.mysql.sql(the data layer)

Nine tables in Third Normal Form, with integrity enforced by the database rather than by application code.
The UNIQUE constraint on reference is what makes BR-10's uniqueness guarantee real: enforced by the schema rather than only by the code that generates the value. Test case TC-41 verifies it by attempting a duplicate insert and requiring the database to refuse.

The statuscolumn is VARCHAR(20)where Phase 4 §4.5.3 specifies an ENUM. That is DEF-06.

```
CREATE TABLE IF NOT EXISTS `viewing_requests` (
  `id`              INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `reference`       VARCHAR(20)  NOT NULL COMMENT 'RF-YYMMDD-XXXX shown to the student',
  `full_name`       VARCHAR(160) NOT NULL,
  `email`           VARCHAR(190) NOT NULL,
  `phone`           VARCHAR(20)  NOT NULL,
  `guardian_email` VARCHAR(190) DEFAULT NULL,
  `campus_id`       VARCHAR(40)  NOT NULL,
  `preferred_date` DATE          NOT NULL,
  `move_in_month`   CHAR(7)      NOT NULL COMMENT 'YYYY-MM',
  `notes`           VARCHAR(500) DEFAULT NULL,
  `status`          VARCHAR(20)  NOT NULL DEFAULT 'new',
  `created_at`      TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uq_reference` (`reference`),
  CONSTRAINT `fk_vr_campus` FOREIGN KEY (`campus_id`)
    REFERENCES `campuses` (`id`) ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5.2.4 Security as implemented

Table 5.2.1 — Security controls and the cases that verify them

Control	Implementation	Verified by
Injection prevention	PDO prepared statements, native prepares, no concatenation	TC-45, TC-46 — six payloads across every parameter
Output escaping	htmlspecialcharswith ENT_QUOTESon every dynamic value	TC-47 — reflected and stored payloads
CSRF protection	32 random bytes per session, compared with hash_equals()	TC-44 — missing, wrong and valid tokens
Error handling	Exceptions caught, logged, replaced with a neutral message	TC-48 — four error paths checked for disclosure
Credential handling	Configuration git-ignored; a sample template committed instead	Repository inspection
Transactional integrity	Both inserts in one transaction, rolled back on any exception	TC-40 — no orphan rows on either side
Data minimisation	No identity numbers, banking details or street addresses	TC-14 — area is suburb level only
Search engine exclusion	noindexon every page	TC-49 — all four pages

5.2.5 Known limitations in the delivered code

Testing revealed seven faults in total. Subsequent to the execution of the suite, which uncovered all seven identifed defects, I was able to correct each of these defects present in the suite. The code you delivered, with them included and corrected at a later stage, is listed here because an account of what was delivered that omitted them is incomplete. Section 5.4.6 describes each as found; 5.4.7 describes correction and run that verified correction.

Table 5.2.2 — Defects found, and their present state

ID	Severity	Summary as found	State
DEF-02	High	A viewing request naming no rooms is accepted and stored	**Corrected**
DEF-01	Medium	A name longer than 160 characters is silently truncated	**Corrected**
DEF-03	Medium	Rooms from another campus are accepted on a request	**Corrected**
DEF-04	Medium	The move-in month is not validated against the offered range	**Corrected**
DEF-05	Medium	An invalid campus is refused by the database rather than by validation	**Corrected**
DEF-07	Medium	The guardian hint promises an email the system never sends	**Corrected**

DEF-06	Low	The status column accepts any value	**Corrected**

Apart from these items, Phase 4 4.8.5 records seven items of outstanding work between this prototype and a system that could responsibly hold real students' personal information: rate limiting on the public form, moving the application files out of the webroot, agreeing a POPIA retention period, HTTPS, a least-privilege DB account, an admin interface, and email delivery.

5.3 Testing

This section documents the verification of EMERGEX: what was tested, how, with which tools, and on what basis a build is judged fit to demonstrate. Section 5.4 presents the test cases themselves, the results of executing them against a dated run, and the defects that execution uncovered.

5.3.1 Objectives

1. Verify that every implemented requirement behaves as Phase 3 specified.
2. Verify that data written through the interface is correct and complete in the database, rather than merely plausible on screen.
3. Verify that the security controls resist ordinary hostile input.
4. Establish a repeatable regression suite, so a later change cannot silently undo an earlier fix.
5. Record every defect found, with enough detail to reproduce and correct it.

5.3.2 Scope

In scope

- All four pages: landing, search, listing detail, viewing request
- The complete journey: browse, filter, compare, shortlist, estimate cost, request a viewing, receive a reference
- All ten functional requirements and all fourteen business rules
- Security controls: CSRF, injection resistance, output escaping, error disclosure, search-engine exclusion
- Accessibility and responsiveness at seven viewport widths
- Behaviour with no database connected

Out of scope

- **Load and performance testing.** The prototype runs on one developer machine against eighteen seeded listings; a throughput figure measured there would not be meaningful.
- **Cross-browser testing.** Execution used Chromium only.
- **Penetration testing.** The security layer verifies that specific controls hold; it is not an adversarial assessment.
- **Assistive-technology testing.** Accessibility verification was structural — bound labels, keyboard reachability, live regions — not a screen-reader audit.

5.3.3 Test strategy — the six levels

Testing is organised into six levels. Each answers a different question, and a defect that escapes one level should be caught by the next.

Table 5.3.1 — Test levels

Level	Question it answers	What it would catch	Test functions
UI	Does the interface render correctly and accessibly?	A badge that does not appear; horizontal overflow on a phone	40
Functional	Does the application follow its own rules?	A past date accepted; a filter returning the wrong rooms	73
Integration	Did what the browser did actually reach MySQL?	A confirmation shown when nothing was stored	37
Security	Do the controls hold under hostile input?	An injection payload treated as SQL rather than as text	19
Logic	Is the arithmetic right? (no browser needed)	A discount applied to the deposit as well as the rent	13
Regression	Has a previously correct behaviour been lost?	A security header silently dropped in a later change	8
End-to-end	Do complete journeys work across pages?	A shortlist that does not carry into the booking form	5
Distinct test functions			**149**
Executions in a full run			**230**

> **Two figures, measuring two things.** The suite contains **148 test functions**, which produce **229 executions** when run. The difference is parameterisation: one function checking the phone-number rule runs against six different numbers and is counted once as a function and six times as executions.
>
> The level totals add to more than 148 because a test may carry more than one marker. A test proving a deactivated listing disappears from search is both integration and regression, and is counted in both without being duplicated in code.
>
> These figures are produced by `tools/layer_index.py`, which reads the markers back out of the suite rather than being maintained by hand. The full index, listing which tests belong to which level, is written to `reports/layer-index.md` — so the numbers in this table cannot drift from the suite they describe.

It is concern of integration. A browser test can prove only what is seen on-page. There is nothing to prove by a page that says your request has been saved. After all, the page would have printed the same whether or not the insert silently failed. The only way to prove persistence actually occurred is by testing againts MySQL directly in an intergation test.

Relationship to earlier phases

Table 5.3.2 — V-model mapping

Development phase	Test level	Verified by
3.6 Functional requirements (FR1–FR10)	Acceptance	End-to-end, functional
3.7 Non-functional requirements (NFR1–NFR8)	Security, accessibility	Security, UI
3.4 Business rules (BR-01 to BR-14)	Component	Functional, logic
4.3 Architectural design	Integration	Integration
4.5 Database design	Integration	Integration
4.7 Interface design	Interface	UI

5.3.4 Test environment

Table 5.3.3 — Environment specification

Component	Specification
Operating system	Windows 11
Web server	Apache (XAMPP), localhost
Application runtime	PHP 8
Database	MySQL 8, schema richfield_accommodation, port 3306
Browser under test	Chromium, Playwright-managed build, headless
Test runtime	Python 3.13.7
Test data	Eight campuses, eighteen listings, ten amenities, five room types, loaded by database/seed.php

The suite begins in a known state because the seed script is re-run before a recorded execution. The reason for this importance is that multiple tests issue view requests and temporarily modify the data within the listing, causing the assertions on the count to drift with each run when we do not have a deterministic value to start with.

5.3.5 Tools, and why they changed

Table 5.3.4 — Tooling

Tool	Version	Purpose
Python	3.13.7	Test implementation language
pytest	8.3.4	Test framework: fixtures, markers, parameterisation
Playwright for Python	0.6.2	Browser automation — drives a real Chromium instance
PyMySQL	1.1.1	Direct database access for the integration layer
requests	2.32.3	Posting directly to the server, bypassing the browser
pytest-html	4.1.1	Browsable run report
Visual Studio Code	—	Authoring and interactive execution
BeyondTheBugs	—	Test case management, built by and owned by a single member

The change from C# to Python

An earlier version of this suite was written in C# with NUnit and Playwright for .NET. It had to be rebuilt, and the opportunity was taken to reconsider the language. Both parts of that are worth stating plainly.

> **Why it was rebuilt.** Early in the project the group had two prototypes under development. The first test suite was written against one of them; the group subsequently confirmed that the Richfield accommodation interface is the system being delivered. The suite therefore tested an application that was not the deliverable, and was rewritten against the correct one.
>
> This was discovered by the group during Phase 4, with time to correct it, rather than at submission. The test cases were re-derived from the Phase 3 requirements at the same time, which is why every case in this phase carries the requirement it verifies.

Since the suite was being rewritten in any case, four technical reasons favoured Python over C#:

- **No build step.** The C# suite had to compile before anything ran. pytest executes the source directly, which materially shortens the edit-run loop when iterating on selectors.
- **Parameterisation is more concise.** One @pytest.mark.parametrize turns a single function into eight executions. The NUnit equivalent is considerably more ceremony for identical coverage. Of the 229 tests in this suite, roughly 70 are parameterised expansions of a smaller number of functions.
- **Lower barrier for the group.** Python and Visual Studio Code need no Visual Studio installation, so any team member can run the suite.
- **The Playwright knowledge transferred.** Locators, web-first assertions and browser contexts work the same way in both, so the move cost the mechanics of rewriting rather than the understanding.

Test case management — BeyondTheBugs (BeyondTheBugs.netlify.app)

The sixty test cases are managed in **BeyondTheBugs**, a test management tool built for this project rather than bought. Commercial tools such as Jira or TestRail were not available under licence, and building the tool meant establishing from first principles what a test case record must hold: preconditions, numbered steps, an expected result, a lifecycle status, and the requirement it traces to.

Table 5.3.5 Accessing the test management tool

Item	Detail
Address	https://beyondthebugs.netlify.app/
To open it	Visit the address above and select **Open Tool**. No sign-in is required.
Built by	Novithan Naidoo (402412304)
Hosting	Netlify, deployed from GitHub. Data is held in a hosted Postgres database and mirrored to browser storage, so the tool keeps working offline
Holds	The sixty cases for this project, both runs with a result against every case, and the seven defects linked to the cases that found them

The tool records your cases and organises them on a board by Status. You can group them into dated Test Runs, record a Result per case per run, and link back defects to the case that caused it. In this stage it acquired three capabilities that this project needed. JSON import and export, so the sixty cases can move between the tool and version control rather than being typed in one at a time. Also, a trash that holds deleted cases until they are permanently removed. Moreover, a search that matches on the case reference and on the requirement tags, so TC-37 or BR-06 finds what it should.

Keeping the specifications in the tool and the scripts in the repository is deliberate. A test case is a specification the whole group can read and agree on. A test script is software, and belongs under version control beside the code it exercises.

5.3.6 Framework architecture

The suite is not a flat collection of scripts. Shared machinery lives in a support package, separate from the tests themselves.

```
tests/
├── pytest.ini          Markers, report settings
├── requirements.txt    Pinned versions
├── conftest.py         Shared fixtures
│
├── support/
│   ├── config.py       Every environment value and seeded constant
│   ├── database.py     Direct MySQL access for the integration layer
│   └── pages.py        Page objects — EVERY selector lives here
│
├── suites/
│   ├── 00-smoke/        test_smoke.py          wiring check
│   ├── 01-ui/           test_campuses.py       TC-01 to TC-07
│   │                    test_behaviour.py      TC-50 to TC-56, TC-59, TC-60
│   ├── 02-functional/   test_search.py         TC-08 to TC-13
│   │                    test_listings.py       TC-14 to TC-18
│   │                    test_shortlist.py      TC-19 to TC-22
│   │                    test_validation.py     TC-30 to TC-38
│   ├── 03-logic/        test_calculator.py     TC-23 to TC-29
│   ├── 04-integration/  test_submission.py     TC-39 to TC-43
│   ├── 05-security/     test_security.py       TC-44 to TC-49, TC-57, TC-58
│   └── 06-e2e/          test_journeys.py       complete journeys
│
├── tools/layer_index.py     Generates the by-layer index
├── tools/Organise-By-Layer.ps1
└── TestCases/               The sixty specifications, and the run record
```

The name of each folder corresponds to the level that the files within the folder mostly cover plus a README. Two levels have no folder because a test may belong to more than one level; moreover, a folder can hold a file only once. The eight regression tests exist to guard behaviours that live elsewhere. Moving them would separate each guard from the thing it guards.

The markers remain the accurate record. pytest -m integration returns all 37, wherever they are filed, and each folder's README names the other levels its tests also cover.

Table 5.3.6 — Framework components

Component	Responsibility
config.py	Base URL, credentials, seeded constants and business-rule values in one place. Every value is overridable by environment variable, so the suite runs on another machine without code changes.
database.py	Parameterised MySQL access for the integration layer, plus cleanup helpers. Every statement is parameterised — the suite must not model bad practice while testing for it.

pages.py	Page objects. **Every CSS selector in the suite lives here and nowhere else.**
conftest.py	Shared fixtures, and a session check that stops the run with one clear message if Apache is not running, rather than letting a hundred tests fail with navigation errors.

> **Why the Page Object Model matters.** Tests express intent — open the search, filter by rent, submit the form — while page objects hold the knowledge of which element that means. When markup changes, the fix is one line in one file rather than a search across 229 tests.

> This was not theoretical. During development the listing-name selector was wrong in a way that failed five tests at once; correcting a single method in `pages.py` fixed all five.

5.3.7 Test design principles

- **No selector appears in a test.**
- **No fixed waits.** Playwright's assertions retry until a timeout, so there is not one sleepin the suite.
- **Tests clean up after themselves.** Every created viewing request is deleted; every altered row is restored.
- **Negative cases are first-class.** Roughly half the suite asserts that something is correctly refused. A suite that only proves the happy path proves very little — and every one of the seven defects found lies on a negative path.
- **Integration means the database.** Integration tests assert on MySQL rows.
- **Prove derivation, not coincidence.** Where a figure is claimed to be computed from data, the test changes the data, reloads, and requires the figure to follow. Matching once could be a number that happened to be correct when it was typed in.
- **Tests that document defects are kept.** Seven tests assert correct behaviour the build does not implement and therefore fail. Deleting them would raise the pass rate and conceal seven real defects.

5.3.8 Entry and exit criteria

Table 5.3.7 — Entry and exit criteria

Entry criteria	Exit criteria
• Application reachable at the configured address • Database schema imported and reference data • seeded Browser binaries installed • The smoke suite passes	• All sixty cases executed or explicitly recorded as blocked • Every failure traced to a documented defect or a test fault • No unexplained failure remaining

	• Results written to file, not only to screen

5.3.9 Executing the suite

The whole suite runs with one command from the testsfolder:

```
pytest
```

This writes a browsable HTML report and a JUnit XML result file to reports/. Individual levels can be run on their own, because the layer is recorded as a marker rather than as a folder:

```
pytest -m security          # every security test, wherever it lives
pytest -m "ui or logic"     # combine levels
pytest --lf                 # only what failed last time
pytest --headed --slowmo 500   # watch it work in a visible browser
```

The last of these is worth noting for demonstration purposes: the suite can be run with the browser visible and deliberately slowed, so an observer can watch it select campuses, apply filters, fill the form and read results.

5.4 System Testing

5.4.1 Test case design

Sixty test cases were designed. They are listed in summary in 5.4.4 and in full in Appendix A.

The requirements were used to derive the cases, not the code. The ten functional requirements, eight non-functional requirements and fourteen business rules lead to thirty-two testable statements. Some tests demand multiples cases for proper coverage, especially validation, whereby each field and each boundary requires its own.

The selection was carried out through risk, using four criteria, namely in order, impact if it fails, frequency of use, conditional logic is where defects cluster, likely to break, and invisibility, missing noindex directive fails silently, only automation catches it.

It matters to derive cases from requirements rather than from code, it produced the result in 5.4.6. A case written by reading apply.php would have tested the validation that's there. Written from BR-06 case.

The case "a viewing request must name at least one listing" confirms the validation is there, and fails when it is not.

5.4.2 Case status and execution result

Two distinct pieces of state are tracked for every case, and conflating them is a common source of confusion.

Table 5.4.1 — Case status versus execution result

Concept	What it records	Values used
Case status	The lifecycle of the specification — has this case been written, reviewed and agreed?	draft→ ready→ approved
Execution result	What happened the last time the case was run, in a particular run	passed, failed, blocked, pending

A case can be approved and still fail: approval means the specification is correct, not that the application satisfies it. All sixty cases carry the status **approved**, and seven of them failed.

By keeping the two separate, history also preserved. A result is specific to a run. So if a case passes today then fails tomorrow after my change the case has two results not one value that gets overwritten. The test management tool sees runs as a first-class concept instead of simply storing a single result field on every test case.

5.4.3 The test runs of record

There are two. **TR-001** is the execution that found the defects, and is the run described in sections 5.4.4 to 5.4.6.

TR-002 is the execution that verified their correction, and is described in section 5.4.7.

Table 5.4.2 — Test run TR-001

Field	Value
Run identifier	**TR-001**
Date executed	**1 November 2026**
Executed by	Novithan Naidoo (402412304), Group 55
Build under test	Richfield accommodation interface, local XAMPP deployment
Environment	Windows 11 · Apache · PHP 8 · MySQL 8 · Chromium headless
Preconditions	Reference data restored with database/seed.php; Apache and MySQL restarted
Scope	All sixty cases
Command	pytest
Duration	**6 minutes 42 seconds** (402.27 seconds)
Evidence	reports/TR-001/report.html, reports/TR-001/results.xml

A test run means a single dated execution of a set defined cases against one build in one environment It's what carries the results: the cases are reusable, the run is the record of what they proved on a given day. Running the suite after doing correction leads to TR-002. TR-001 remains unchanged as evidence of prior state.

Where the results are recorded

Both runs are recorded in **BeyondTheBugs**, the test management tool built for this project, alongside the sixty case specifications and the defects raised against them.

Table 5.4.3 — The test management record

Item	Detail
Address	https://beyondthebugs.netlify.app/— select **Open Tool**. No sign-in required.
Test cases	All sixty, each with preconditions, numbered steps, an expected result, a lifecycle status and the requirement it verifies
Test runs	TR-001 (defects found) and TR-002 (corrections verified), each carrying a result against every case
Defects	DEF-01 to DEF-07, each linked to the case that found it
Board	Cases arranged by lifecycle status, draft through approved

The distinction in Table 5.4.1 becomes practicable through the instrument in use. A single specification and lifecycle status are held by a case; a run holds its own result for that case. TC-37 is hence registered once as a specification, failed in TR-001, passed in TR-002 that is a triple of facts not overwriting one another. Only a single resultant field on each case could not express that and the history of what has been found would be lost the moment that it is fixed.

> **A first attempt was discarded, and it is recorded here rather than omitted.** MySQL had not restarted alongside Apache, so roughly thirty integration tests skipped rather than running. A run in which the database layer is untested is not evidence of anything. The service was started and the suite re-run from the beginning; TR-001 is that second, complete execution.
>
> It is worth stating because the alternative — quietly reporting the second attempt as though it were the first — would make the record less trustworthy without making it better.

5.4.4 Results

Table 5.4.4 — Execution summary, TR-001

Measure	Value
Test cases in scope	60
Cases passed	**50**
Cases failed	**7**
Cases blocked	**3**
Test functions	149
Executions collected	230
Executed	224
Passed	**217**
Failed	**7**
Skipped	6
Pass rate (of executed)	**96.9%**
Duration	6 minutes 42 seconds
Failures traced to a documented defect	7 of 7 (100%)
Unexplained failures	**0**

The three blocked cases, TC-54 to TC-56, check the FR10 by moving the app’s config file aside to show the site runs without a DB. They are built in and can be invoked at will, but they are not part of an ordinary run: an interrupted run would leave the place on sample data. When tests are not run, they don’t provide any information about whether it

works or not; therefore, they are recorded as blocked.

Table 5.4.5 — Where the failures fall, by test level

Level	Test functions	Failing	Failing cases
UI	40	1	TC-60
Functional	73	4	TC-31, TC-37, TC-38, TC-60
Integration	37	1	TC-59
Security	19	2	TC-57, TC-58
Logic	13	0	—
Regression	8	0	—
End-to-end	5	0	—

The totals are greater than 149 as a test may hold more than one marker, TC-60 is reported under both UI and functional for that reason. It is worth noting that no failures at all were seen at two levels – logic and end-to-end recorded. This means that the cost calculation and the complete student journey behave exactly as specified.

Table 5.4.6 — Case results by area

Area	Cases	Outcome
Campuses and the landing page	TC-01 – TC-07	**All passed**
Search and filtering	TC-08 – TC-13	**All passed**
Listing detail	TC-14 – TC-18	**All passed**
Shortlist	TC-19 – TC-22	**All passed**
Cost estimator	TC-23 – TC-29	**All passed**
Form validation	TC-30 – TC-38	**3 failed** — TC-31, TC-37, TC-38
Submission and persistence	TC-39 – TC-43	**All passed**
Security controls	TC-44 – TC-49	**All passed**
Accessibility and responsiveness	TC-50 – TC-53	**All passed**
Fallback behaviour	TC-54 – TC-56	**Blocked — not executed**
Phase 3 integrity findings	TC-57 – TC-60	**All four failed**

5.4.5 Traceability

Every case carries the requirement it verifies, recorded as a tag on the case and in the test's own documentation. The matrix below is therefore derived from the cases rather than assembled separately, which means it cannot drift out of date.

Table 5.4.7 — Requirement coverage

Requirement	Statement	Cases	Result
FR1	Display every campus with location and bed count	TC-01 – TC-07	**Verified**
FR2	Search by campus, rent, room type and amenities	TC-08 – TC-13	**Verified**
FR3	Show rent, type, status, amenities and travel times	TC-14 – TC-18	**Verified**
FR4	Plot listings and campus on a map	TC-10 (map container)	**Verified**
FR5	Shortlist up to four, kept across reloads	TC-19 – TC-22	**Verified**
FR6	Accept and persist a viewing request	TC-39 – TC-43	**Verified**
FR7	Validate all input server-side	TC-30 – TC-38, TC-57, TC-58	**Defects found**
FR8	Estimate the total move-in cost	TC-23 – TC-29	**Verified**
FR9	Protect the form against CSRF	TC-44	**Verified**
FR10	Keep working with no database connected	TC-54 – TC-56	**Not executed**
NFR2	Every query parameterised	TC-45, TC-46	**Verified**
NFR4	WCAG 2.2 AA, keyboard operable	TC-51 – TC-53	**Verified**
NFR5	Never present sample data as real or claim an affiliation not held	TC-02, TC-04, TC-15, TC-43, TC-49, TC-60	**Defect found**
NFR6	No horizontal overflow, 360–1920 px	TC-50	**Verified**
NFR8	Output escaped, errors logged not shown	TC-47, TC-48	**Verified**
BR-01	Only active listings appear	TC-18	**Verified**
BR-02	Inspected status shown exactly as stored	TC-15	**Verified**
BR-03	Unconfirmed campus locations labelled	TC-02	**Verified**
BR-04	Each listing belongs to one campus and room type	TC-08, TC-58	**Defect found**
BR-05	At most four shortlisted listings	TC-20	**Verified**
BR-06	A request must name at least one listing	TC-37	**Not enforced**
BR-07	Rooms must be active and at the selected campus	TC-38	**Not enforced**
BR-08	Phone must be a South African mobile number	TC-33	**Verified**
BR-09	Preferred date cannot be in the past	TC-35	**Verified**
BR-10	Unique server-generated reference	TC-39, TC-41	**Verified**
BR-11	A request and its rooms saved together or not at all	TC-40	**Verified**
BR-12	Only data needed to arrange a viewing is collected	TC-14, TC-56	**Verified**
BR-13	Move-in cost formula	TC-23 – TC-29	**Verified**

BR-14	Location shown at suburb level only	TC-14	**Verified**

5.4.6 Defects identified

The TR-001 had a total of seven defects. As many as 21 of these correspond to a finding the group wrote up in phase 3 3.4.5 during data analysis, before there was any test. The defect register below describes the defect as found, and the correction specified for it; section 5.4.7 records the corrections as applied, and the run that verified them.

Table 5.4.8 — Defect register, as raised in TR-001

ID	Severity	Case	Rule	Description	Correction specified
DEF-02	High	TC-37	BR-06	A viewing request naming no rooms is accepted and stored. The run produced reference RF-261101-3591 with **zero rows** in viewing_request_rooms. The provider receives a name, a phone number and a date, with no indication of what the student wants to see; the student receives a reference and waits for a call that cannot usefully be made.	Reject the submission when no room is selected, with the error beside that section (Phase 4 §4.6.1)
DEF-01	Medium	TC-31	FR7	A name longer than 160 characters is accepted and **silently truncated** to fit the column. A 200-character name was stored as 160. Neither the student nor the provider is told anything was lost, so the provider calls using a name that is not the student's.	Add a maximum-length check before the transaction opens (Phase 4 §4.6.1)

DEF-03	Medium	TC-38	BR-07	Rooms named on a request are checked for existence but not for location. The run accepted a request for campus cape-town naming acacia-court-bryanston — approximately 1 400 km away. The request routes to the wrong campus.	Resolve each slug and compare its campus with the submitted campus (Phase 4 §4.6.1)
DEF-04	Medium	TC-57	FR7	The move-in month is not validated. A value of 1999-01, posted directly to the server, was stored unchanged. Not reachable through the interface, because the field is a dropdown — which is precisely why it was found by posting rather than by clicking.	Validate against the generated twelve-month list (Phase 4 §4.6.1)
DEF-05	Medium	TC-58	FR7, BR-04	The campus is not validated. An unknown value reaches the insert, the foreign key rejects it, the exception is caught, and the student is	Validate against the campus table in the validation sequence (Phase 4 §4.6.1)

				told "We could not save your request just now" with no indication of which field is wrong. The data is protected; the student is not helped.	
DEF-07	Medium	TC-60	NFR5	The hint beneath the guardian email field reads "They receive the same confirmation and lease summary you do." No email is sent by this release. The same claim appears in the For Parents section of the landing page.	Reword to describe what the system does (Phase 4 §4.7.2)
DEF-06	Low	TC-59	FR6	The statuscolumn is VARCHAR(20)and accepted not-a-real-status. It is meant to hold one of four values. Low severity because nothing reads the column yet.	Change to ENUM('new','contacted','viewed','closed') (Phase 4 4.5.3)

The defect that matters most

DEF-02 is the significant one, and it is worth setting out in full because it demonstrates what the three phases were for.

Business rule BR-06, written during Phase 3, states that a viewing request must name at least one listing. Phase 3 §3.4.5 finding 1 recorded that the prototype did not enforce it, and identified the risk as "requests the campus cannot act on". Phase 4 §4.6.1 specified the correction — a room-count check before the transaction opens — and Phase 4 Figure 4.15 captured the form showing validation errors on four fields and nothing beside "Rooms to view".

Test case TC-37 asserts the submission is refused. In TR-001 it failed, and the failure produced a stored artefact: reference RF-261101-3591, a complete viewing request with no rooms attached.

A defect predicted during analysis, specified for correction during design, and proven by executed evidence during implementation. That is the thread the three phases were meant to produce.

Why these defects were found

None of the seven would plausibly have been found by manual testing.

A person filling in the form naturally selects a room before submitting, so they would never discover that leaving it empty is accepted. Nobody types a 200-character name by hand. The campus and move-in month are dropdowns, so a person using the interface *cannot* submit an invalid value — those two were found only because the tests post directly to the server, bypassing the browser entirely.

That last point generalises into the most useful thing this phase established:

> **Client-side validation is a usability feature. It is never a security control, because the attacker chooses the client.**
>
> This was learned the hard way. An early injection test appeared to pass when driven through the interface; investigation showed the browser was refusing to submit the malformed value, so the request never

> reached PHP. The test was proving that Chromium validates input, not that the application resists injection. Rewritten to post directly, it exercised the server and passed on its merits.
>
> Any security testing conducted exclusively through the interface will report false passes.

5.4.7 Corrections, and verifying them (TR-002)

Finding a defect is half of the work. All seven were corrected, and the suite was executed a second time to prove it — because a correction that is only inspected is a claim, and a correction that is re-tested is evidence.

What was changed

Table 5.4.9 — Corrections applied

ID	Case	File changed	Change made	TR-002
DEF-02	TC-37	apply.php	The submission is refused unless at least one room is named, and the error is rendered beside the "Rooms to view" section rather than at the top of the form, so the student is told which part is wrong	**Pass**
DEF-01	TC-31	apply.php	A maximum length of 160 characters on the name, checked before the transaction opens, matching the column it is stored in	**Pass**
DEF-03	TC-38	apply.php	Each submitted slug is resolved and its campus compared with the campus on the request; a mismatch is refused with an explanatory message	**Pass**
DEF-04	TC-57	apply.php	The move-in month is validated against the twelve months the form itself generates, so a posted value outside that set is refused	**Pass**
DEF-05	TC-58	apply.php	The campus is validated against the campus table during validation, so an unknown value produces a message beside that field instead of reaching the insert and being caught as a database error	**Pass**
DEF-07	TC-60	apply.php, index.php	The guardian hint reworded to describe what the release actually does, and the matching claim on the landing page corrected with it	**Pass**
DEF-06	TC-59	schema.mysql.sql	statuschanged from VARCHAR(20)to ENUM('new','contacted','viewed','closed'), applied to the live database with ALTER TABLE	**Pass**

Six of the seven are validation rules in one file, which is itself a finding: the defects were not scattered through the system but concentrated at the single point where untrusted input is accepted. That is the place worth testing hardest.

The verification run

Table 5.4.10 — TR-001 compared with TR-002

Measure	TR-001 — before	TR-002 — after
Date executed	1 November 2026	1 November 2026
Test functions	149	149
Executions collected	230	230
Executed	224	224
Passed	**217**	**224**
Failed	**7**	**0**
Skipped	6	6
Pass rate (of executed)	96.9%	**100%**
Duration	6 minutes 42 seconds	7 minutes 48 seconds
Open defects	**7**	**0**
Evidence	reports/TR-001/	reports/TR-002/

The six skips are unchanged between the runs: they are the opt-in fallback cases TC-54 to TC-56, excluded from an ordinary execution for the reason given in section 5.4.4. Nothing else in the suite was modified between the two runs except where noted below, so the difference in results is attributable to the corrections.

Correcting the defects exposed a second problem

The first attempt at TR-002 produced **thirty failures**, and every one reported the same thing: *"All the rooms you select must be at your chosen campus."*

The cause was in the tests, not the application. The helper that fills in a valid request selected a room at Umhlanga while leaving the campus field at its default of Bryanston. Before DEF-03 was corrected the application accepted that combination, so the suite had been submitting an impossible request throughout and nothing objected. Enforcing BR-07 made those submissions invalid — correctly — and thirty tests said so within eight minutes. The fixtures were changed to select rooms at the campus being submitted, which is also what a student does.

Two things follow from this, and both are worth more than the defect itself.

The first is that the regression layer demonstrably works. One change to one file, thirty immediate reports, one cause, located in minutes. That is the return on having written the tests, and it is a return that cannot be claimed until something actually changes.

The second is that **an unenforced rule can conceal unrealistic test data**. The suite was passing while submitting a request no student could make. It passed because the application was wrong in the same direction as the tests. A green run is evidence that the system behaves as the tests expect, which is not the same as evidence that the tests expect the right thing.

Two further findings, in the suite rather than the system

A flaky test. One fixture selected "the campus with the most rooms" using ORDER BY COUNT(*) DESC LIMIT 1. Several campuses hold the same number of rooms, and MySQL is under no obligation to break that tie the same way twice, so the fixture returned different campuses on different runs and a small group of tests passed or failed accordingly. Explicit tie-breaking columns were added. This is recorded because a flaky test is worse than a failing one: a failing test is read, while a flaky test teaches the reader to disregard results.

A false pass. TC-59 continued to fail after the status column had been correctly changed to an ENUM. The test waited for the write to be refused, but MySQL in this environment does not run in strict mode: an invalid enumeration value is coerced to an empty string with a warning rather than rejected. The constraint was working; the test was watching the wrong signal. It now asserts on the value that was actually stored, which gives the same answer on a strict server and on a permissive one.

A second false pass, of the same family. TC-58 originally passed, and should not have. Its assertion accepted any sign of refusal, and the request *was* being refused — by the database's foreign key, after validation had let the value through. The test was reporting success for exactly the behaviour it existed to detect. It was tightened to distinguish a validation message from the application's generic "we could not save your request", and it then failed, as DEF-05 required. An assertion loose enough to pass for the wrong reason is indistinguishable from no assertion at all.

And one about the environment. Until this point the suite was exercising a copy of the application held directly under htdocs, not the copy inside the repository — the copy that is submitted. The two were almost certainly identical, but nothing kept them so, and an edit to the delivered copy would not have been tested. The base address now points at the repository. **Test what you ship**, or the suite eventually certifies something nobody receives.

5.4.8 Evaluation of the testing results

Overall assessment

The application is functionally sound and securely implemented. In TR-001, fifty of sixty cases passed, including every case covering search, filtering, listing detail, the shortlist, the cost estimator, submission and persistence. The seven that failed were all corrected, and in TR-002 **fifty-seven of sixty cases passed, with none failing** and the remaining three recorded as blocked for the reason given in section 5.4.4.

The assessment that follows is written against TR-001, because TR-001 is where the evidence was produced. A clean run states that nothing was found; it is the run that found things which explains what the suite is worth.

The **security controls hold**, which is worth stating plainly because failures are easier to notice than passes. Six distinct injection payloads were submitted against every search parameter, the listing slug and the form, and each was treated as literal text: no database error was disclosed, no payload acted as a wildcard, and the schema was unchanged. A script tag submitted through the form and a script tag stored in a listing name both rendered as text rather than executing. A submission with its CSRF token removed was refused, as was one with an altered token, while a submission with a valid token succeeded — so the control is effective without being obstructive. No error response disclosed a file path, a stack trace or a SQL error.

The **cost estimator is arithmetically correct** across all three payment plans, including the rule that carries the most meaning: the discount applies to rent only and never to the deposit or the administration fee. That was verified by reading the deposit under all three plans and requiring the three to be identical.

The **write path is sound**. Every field survives the round trip unaltered, both tables are written inside one transaction, references are unique and enforced as such by the schema, and the confirmation page claims storage only when a row genuinely exists.

Where the defects cluster

All seven defects lay in one place: **the validation sequence of the viewing-request form**, and the schema constraint behind it. Nothing was found in the read paths, the calculations, the security controls or the presentation layer — and the corrections in section 5.4.7 touch the same single file, which is the clearest confirmation of the diagnosis.

That concentration is itself a finding. The same codebase validates the search page's filters carefully — the campus must resolve, the rent is clamped, unknown room types and amenities are discarded — while the form's campus and month fields are passed through unchecked. Two different standards in one application, separated by about two hundred lines of code.

The likely reason is that the search filters were written knowing the values come from a URL, while the form fields were written knowing they come from a dropdown. The dropdown constrains the browser, not the request, and that distinction is the root of DEF-04 and DEF-05.

What the suite established beyond finding defects

- **Figures on the landing page are genuinely computed.** The test changes a campus bed count, reloads, and requires the displayed total to move by the same amount. A hardcoded number would not.
- **The inspected badge is drawn only from stored data.** Setting the flag makes the badge appear; clearing it makes the badge go.
- **Travel times are correct, not merely present.** The suite implements the haversine calculation independently and compares.
- **Amenity filtering is AND rather than OR.** Proven by requiring the result count to narrow monotonically as amenities are added.
- **Deactivating a listing is a soft delete.** The room vanishes from search and its URL returns 404, while the row remains for history.

A note on the suite's own defects

The test suite is software and had its own faults. Four are recorded in section 5.4.7 — a flaky fixture, two assertions that passed for the wrong reason, and a base address pointing at the wrong copy of the application — because they surfaced during the correction cycle rather than during the first run.

They share a shape worth stating on its own. None of them produced a red result. Each made the suite report success it had not earned, and each was found only because something else changed and forced the question. **A test that passes for the wrong reason is the hardest fault to notice**, because nothing looks wrong — which is why the figures in Table 5.4.10 are reported alongside an account of what the suite got wrong, rather than on their own.

5.4.9 Limitations

- **One browser.** Execution used Chromium only; rendering differences in Firefox or Safari would not be detected.
- **Seeded data volumes.** Eighteen listings across eight campuses. Pagination, query performance and index effectiveness at realistic volumes are untested.
- **No concurrency testing.** The booking transaction has not been tested under simultaneous requests.
- **FR10 not executed in this run.** The fallback cases are implemented but were recorded as blocked.
- **Accessibility verification was structural.** Bound labels, keyboard reachability and live regions were checked; no screen-reader or colour-contrast audit was performed, so NFR4 is partly rather than fully verified.

5.4.10 Recommendations

The seven defects have been corrected and verified, so the recommendations below concern what should happen next rather than what this phase left undone.

1. **Execute the fallback cases once** so FR10 has a recorded result rather than a blocked one. They are implemented and need only a deliberate run, on a copy of the data, with the configuration restored afterwards.
2. **Keep one copy of the application.** The duplicate outside the repository has been deleted, and the smoke suite was re-run afterwards to confirm nothing depended on it. The practice is worth keeping: while two copies exist, "it passed" is ambiguous about which one passed.
3. **Run the suite before every submission, not after every change.** Eight minutes is short enough to be habitual and long enough that it will be skipped if it is only remembered when something feels wrong.
4. **Validate on the server for every new field.** The two defects that reached the database were both a dropdown value trusted because it came from a dropdown. Any field added later will be added the same way unless the rule is written down, which is why it is stated as a principle in section 5.4.6 rather than only as a fix.
5. **Extend coverage where section 5.4.9 says it stops** — a second browser, realistic data volumes, concurrent submissions, and an accessibility audit that is more than structural.
6. **Address the outstanding work in Phase 4 §4.8.5** before the system holds any real student's data — rate limiting and moving the application files above the web root being the first two.

5.5 Installation

5.5.1 Requirements

Table 5.5.1 — Prerequisites

Component	Requirement
Operating system	Windows 10 or 11; the procedure also works on Linux and macOS with equivalent paths
Web server and runtime	XAMPP with Apache and PHP 8.0 or later
Database	MySQL 8 or MariaDB, included with XAMPP. SQLite is also supported
PHP extensions	pdo, pdo_mysql. mbstringis used when present but is not required
Browser	Any current browser
For the test suite	Python 3.9 or later
Disk space	Approximately 50 MB for the application, plus 400 MB for the test environment and browser

5.5.2 Installing the application

1. **Install XAMPP** to C:\xampp.
2. **Copy the application folder** into the web root, so it sits at C:\xampp\htdocs\Richfield accommodation interface.
3. **Start Apache and MySQL** from the XAMPP control panel and confirm both show as running.
4. **Create the database.** Open http://localhost/phpmyadminand import database/schema.mysql.sql. The script creates the schema itself.
5. **Create the configuration:**

   ```
   copy includes\config.sample.php includes\config.php
   ```

   The supplied values match a standard local XAMPP installation. Edit the username and password only if yours differ from root with a blank password.
6. **Load the reference data:**

   ```
   C:\xampp\php\php.exe database\seed.php
   ```

The script creates any missing tables and loads the campuses, listings, amenities, room types and FAQs. It is safe to re-run: it updates existing rows rather than duplicating them, and never touches submitted viewing requests.

7. **Open the application** at http://localhost/Richfield%20accommodation%20interface/

> **The application also runs without any of steps 4 to 6.** With no configuration file it falls back to the sample data in `includes/data.php`, and every page works — the viewing form validates but stores nothing, and says so. This is FR10, and it exists so that a team member who has not set up MySQL can still open the site.
>
> For assessment the database should be installed, because it is what the integration tests verify.

5.5.3 Verifying the installation

Each step exercises a different layer, so a failure identifies where the problem is.

Table 5.5.2 — Verification steps

#	Action	Expected result
1	Open the landing page	Renders with styling; the red sample-data banner is visible — confirms Apache and the asset paths
2	Read the statistics strip	Beds, campuses, inspected listings and median walk are shown — confirms the database connection, since these are computed from it
3	Open Find a room	Listing cards appear with rents and walking times, closest first
4	Set Max rent to its lowest and tick three amenities	"Nothing matches yet" with a Clear filters control — confirms filtering and the empty state
5	Open a listing	Rent, room type, amenities, walking and driving times — confirms the detail page and distance calculation
6	Shortlist a room, then reload	The count persists — confirms browser storage
7	Submit the viewing form with everything blank	Red errors beside the failing fields — confirms server-side validation
8	Complete it properly and submit	A reference like RF-261101-A1B2, and the fine print reads "has been saved" — confirms the write path
9	Query the database	SELECT * FROM viewing_requests ORDER BY id DESC LIMIT 1 returns the request just submitted

5.5.4 Installing and running the test suite

The suite lives in tests/, beside the application. Run these once, from that folder:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
playwright install chromium
```

The virtual environment keeps the suite's packages separate from the system Python. playwright install chromium downloads the browser build the suite drives, which is why the test environment needs more disk space than the application.

With Apache and MySQL running, execute the suite:

```
pytest
```

This writes reports/report.html and reports/results.xml. Expect seven failures: they are the documented defects in 5.4.6 and are the suite working as intended.

Table 5.5.3 — Useful variations

Command	Effect
pytest -m security	One test level, wherever those tests live
pytest --headed --slowmo 500	Watch the browser work — useful for demonstration
pytest --lf	Only the tests that failed last time
pytest suites/test_validation.py -v	One file, with each test named
python tools/layer_index.py	Regenerate the by-layer index

Nothing in the suite is hardcoded to one machine. Every environment value is overridable:

```
$env:RF_BASE_URL = "http://localhost:8000"
$env:RF_DB_PASSWORD = "secret"
pytest
```

5.5.5 Troubleshooting

Table 5.5.4 — Common problems

Symptom	Cause	Resolution
Pages load but links give "Not Found"	The folder name differs from the URL being used	Use the folder's actual name in the address. Spaces appear as %20
The statistics strip shows zeros	The schema is imported but the seed has not run	Run php database\seed.php
Listings appear but nothing saves	No configuration file, so the site is on sample data	Copy config.sample.php to config.php. The form's fine print says which state it is in
MySQL will not start in XAMPP	Port 3306 is in use by another MySQL installation	Stop the other service, or change the port in both my.ini and config.php
'php' is not recognised	PHP is not on the system PATH	Use the full path C:\xampp\php\php.exe
The suite stops with "application is not reachable"	Apache is not running	Start it. The suite fails fast on purpose rather than letting every test fail with

		navigation errors
Around thirty tests skip	MySQL is not running, so the integration layer cannot connect	Start MySQL and re-run. A run with the integration layer skipped is not a valid record
The site shows sample data after an interrupted run	A fallback test was stopped between moving the configuration and restoring it	Rename includes\config.php.testing-backup back to config.php

5.5.6 Backup and recovery

Reference data is reproducible from seed.phpand needs no backup. Captured data — the viewing requests — cannot be reproduced and is the only thing whose loss would matter.

```
REM Back up
C:\xampp\mysql\bin\mysqldump.exe -u root richfield_accommodation > backup.sql

REM Restore
C:\xampp\mysql\bin\mysql.exe -u root richfield_accommodation < backup.sql
```

Take a backup before any demonstration. Restoring a dump takes seconds; rebuilding a demonstration state by hand during an assessment does not.

Backups contain personal information and fall under the same retention policy as the live tables. They must never be committed to the repository — .gitignore blocks the obvious filenames, but it cannot stop a rename, so this is a procedural control as much as a technical one.

Conclusion

Phase 5 provided a good, working campus-linked accommodation system along with the proof that it works, and the proof of where it doesn't work.

The application meets all ten functional requirements across four pages and nineteen files, and is based on a normalised nine-table MySQL schema, with integrity enforced in the database, not the application. The group devised a testing tool with which to manage the implementation of 149 test functions producing 230 executions across seven levels. The 230 test cases were derived from the Phase 3 requirements.

On November 1, 2026, Test Run TR-001 executed 224 of these tests, of which 217 passed and 7 failed. All the failures were assigned to a documented defect, not a fault with the test. The rate of passing was 96.9% with zero unexplained failures.

The seven defects are all located in one area the validation sequence of the viewing-request form and each one maps to a finding the group made, during data analysis in Phase 3 3.4.5, months before writing a test. Phase 4 specified a correction for all. Phase 5 showed us that all the information was still necessary, including a stored viewing request, reference RF-261101-3591, which does not name any rooms at all.

Thus, all seven were corrected and the suite was executed a second time. In total 224 cases passed and none failed during test run TR-002 which closes the cycle the three phases describe: weakness identified during analysis, correction specified during design, executed evidence that it was still present, and executed evidence that it is gone. The tests that failed in TR-001 were never a deficiency in the work. They were produced as a result of it and they were acceptance criteria for the subsequent corrections.

The lesson most useful from this whole phase is what the correction cycle taught us and not the application. One of the enforced rules made thirty independent tests fail because the test suite was sending a request that no student would be able to send and the application was accepting it. The Suite was not on the winning side. A green run indicates that the system is behaving as the tests expect; it does not mean that the tests expect the right thing. This is why this report sets out what the suite itself got wrong alongside what it found.

The suite is repeatable, self-cleaning and version-controlled alongside the application, so these findings remain protected as development continues.

Appendix A — Test case specifications

The sixty cases, with the requirement each verify and the result in TR-001. The test management tool and tests/TestCases/test_cases.json contain full specifications - preconditions, numbered steps and expected results.

In TR-002, the seven test cases that failed previously were TC-31, TC-37, TC-38, TC-57, TC-58, TC-59 and TC-60. All these now pass and no other test case has changed result. We do not present the TR-002 column here as it would display Pass for fifty-seven rows and Blocked for three, and the per-case results for both runs are in the test management tool..

Table A.1 — Test case inventory, with TR-001 results (all cases approved)

ID	Title	Level	Verifies	Priority	TR-001
TC-01	Display every campus with its location and bed count	UI	FR1	High	**Pass**
TC-02	Unconfirmed campus locations are labelled as such	UI	FR1, BR-03	High	**Pass**
TC-03	Home page statistics are computed from the data	Integration	FR1, NFR5	High	**Pass**
TC-04	Sample-data banner and non-affiliation notice on every page	UI	NFR5	High	**Pass**
TC-05	Campus list on the page matches the campus table	Integration	FR1	Medium	**Pass**
TC-06	Campus filter options match the campus table	Integration	FR1, FR2	Medium	**Pass**
TC-07	Campus links resolve to a filtered search	Functional	FR1, FR2	Low	**Pass**
TC-08	Search by campus returns only that campus's listings	Functional	FR2, BR-04	High	**Pass**
TC-09	Maximum rent filter excludes listings above the cap	Functional	FR2	High	**Pass**
TC-10	Rent filter is clamped to its permitted range	Security	FR2, FR7	Medium	**Pass**
TC-11	Room type filter returns only the selected types	Functional	FR2	High	**Pass**
TC-12	Amenity filter requires every selected amenity	Functional	FR2	High	**Pass**
TC-13	A search matching nothing shows guidance, not a blank area	UI	FR2, NFR4	High	**Pass**
TC-14	Listing detail shows structured accommodation information	UI	FR3, BR-14	High	**Pass**
TC-15	Inspected badge drawn from the stored verification flag only	Functional	FR3, BR-02	High	**Pass**
TC-16	Walking and driving times are shown and are correct	Functional	FR3	High	**Pass**
TC-17	An unknown listing slug returns HTTP 404	Functional	FR3, NFR8	High	**Pass**
TC-18	A deactivated listing disappears from search and detail	Integration	BR-01	High	**Pass**
TC-19	A room can be added to the shortlist	Functional	FR5	Medium	**Pass**
TC-20	The shortlist is capped at four rooms	Functional	FR5, BR-05	High	**Pass**
TC-21	The shortlist survives a reload and follows between pages	Functional	FR5	Medium	**Pass**

TC-22	Shortlisted rooms are pre-selected on the request form	End-to-end	FR5, FR6	High	**Pass**
TC-23	Monthly plan cost calculation	Functional	FR8, BR-13	High	**Pass**
TC-24	Upfront plan discounts five per cent of rent only	Functional	FR8, BR-13	High	**Pass**
TC-25	Termly plan discounts two per cent and splits into terms	Functional	FR8, BR-13	High	**Pass**
TC-26	The deposit is never discounted	Functional	FR8, BR-13	High	**Pass**

TC-27	The administration fee is fixed and always added	Functional	FR8, BR-13	Medium	**Pass**
TC-28	Boundary values produce sane figures	Functional	FR8, BR-13	Medium	**Pass**
TC-29	Every figure is whole rand in the house format	UI	FR8, BR-13	Medium	**Pass**
TC-30	A name shorter than two characters is refused	Functional	FR7	High	**Pass**
TC-31	A name longer than the stored column is refused	Functional	FR7	Medium	**Fail · DEF-01**
TC-32	An invalid email address is refused	Functional	FR7	High	**Pass**
TC-33	Phone validated against the South African mobile format	Functional	FR7, BR-08	High	**Pass**
TC-34	The guardian email is optional but validated when supplied	Functional	FR6, FR7	Medium	**Pass**
TC-35	A preferred date in the past is refused	Functional	FR7, BR-09	High	**Pass**
TC-36	Notes longer than five hundred characters are refused	Functional	FR7	Medium	**Pass**
TC-37	A request naming no rooms is refused	Functional	FR6, BR-06	High	**Fail · DEF-02**
TC-38	Selected rooms must belong to the selected campus	Functional	FR6, BR-07	High	**Fail · DEF-03**
TC-39	A successful submission returns a reference in the agreed format	Functional	FR6, BR-10	High	**Pass**
TC-40	A request and its rooms are stored together	Integration	FR6, BR-11	High	**Pass**
TC-41	References are unique, enforced by the schema	Integration	BR-10	Medium	**Pass**
TC-42	A new request starts with the correct defaults	Integration	FR6	Medium	**Pass**
TC-43	The confirmation states truthfully whether it was stored	Functional	FR10, NFR5	High	**Pass**
TC-44	A submission without a valid token is refused	Security	FR9	High	**Pass**
TC-45	Search parameters resist SQL injection	Security	NFR2	High	**Pass**
TC-46	The listing slug resists SQL injection	Security	NFR2	High	**Pass**
TC-47	Submitted and stored content is escaped on output	Security	NFR8	High	**Pass**
TC-48	Error responses disclose no internal detail	Security	NFR8	Medium	**Pass**
TC-49	Every page instructs search engines not to index	Regression	NFR5	Medium	**Pass**

	it				
TC-50	No horizontal overflow at any supported width	UI	NFR6	High	**Pass**
TC-51	A skip link is the first focusable element	UI	NFR4	Medium	**Pass**
TC-52	Every form field has a bound label and is keyboard reachable	UI	NFR4	High	**Pass**
TC-53	Validation errors and result counts are announced	UI	NFR4, FR7	Medium	**Pass**
TC-54	The site works with no database connected	Functional	FR10	High	**Blocked**

TC-55	The site falls back when connected to an empty schema	Integration	FR10	Medium	**Blocked**
TC-56	The privacy notice matches what the system actually does	Functional	NFR5, NFR7, BR-12	Medium	**Blocked**
TC-57	The move-in month is validated against the permitted range	Security	FR7	Medium	**Fail · DEF-04**
TC-58	The campus is validated against the campus table	Security	FR7, BR-04	High	**Fail · DEF-05**
TC-59	The status column accepts only its defined values	Integration	FR6	Low	**Fail · DEF-06**
TC-60	The guardian hint does not promise an email never sent	Functional	NFR5	Medium	**Fail · DEF-07**

B.1 Prerequisites

1. Apache and MySQL running under XAMPP
2. The application installed and seeded (5.5.2)
3. Python 3.9 or later on the PATH

B.2 Procedure

From the testsfolder:

```
REM 1. Restore the reference data to a known state
C:\xampp\php\php.exe "..\Richfield accommodation interface\database\seed.php"

REM 2. Prepare the test environment (first time only)
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
playwright install chromium

REM 3. Confirm the suite is wired to the application
pytest suites/test_smoke.py -v

REM 4. Execute the recorded run
pytest

REM 5. Generate the by-layer index
python tools/layer_index.py
```

To include the three fallback cases, which move the configuration file aside:

```
$env:RF_RUN_FALLBACK = "1"
pytest
$env:RF_RUN_FALLBACK = ""
```

B.3 Expected outcome

Against the **corrected** build, which is the build in the repository: **224 passed, 0 failed, 6 skipped**, in approximately eight minutes. That is TR-002, and it is what a fresh run should reproduce. The six skips are the fallback cases.

Against the build *before* the corrections in section 5.4.7: **217 passed, 7 failed, 6 skipped**. That is TR-001, and the seven failures are the defects in Table 5.4.8. It is recorded here so the earlier figures in this report can be reproduced from the repository history rather than taken on trust.

In either case, any failure other than those seven is new and should be investigated before the run is recorded.

B.4 Repository contents

Path	Contents
tests/suites/	The nine test files, TC-01 to TC-60
tests/support/	Configuration, database helpers, page objects
tests/conftest.py	Shared fixtures and the fail-fast environment check
tests/TestCases/test_cases.json	The sixty specifications, the source of record
tests/TestCases/TEST_RUN_TR-001.md	The record of the run that found the defects
tests/TestCases/TR-001_results.md	TR-001 results and the defect register
tests/TestCases/TEST_RUN_TR-002.md	The record of the run that verified the corrections
tests/tools/layer_index.py	Generates the by-layer index
tests/reports/TR-001/	TR-001 HTML and XML artefacts
tests/reports/TR-002/	TR-002 HTML and XML artefacts

References

Republic of South Africa (2013) *Protection of Personal Information Act 4 of 2013*. Pretoria: Government Printer.

World Wide Web Consortium (2023) *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. Available at: https://www.w3.org/TR/WCAG22/ (Accessed: 1 November 2026).

Group 55 (2026) *EMERGEX Phase 3 — Analysis*. IT Project 700, Richfield Graduate Institute of Technology. Group 55 (2026) *EMERGEX Phase 4 — System Design*. IT Project 700, Richfield Graduate Institute of Technology.
Project source: Group 55 repository emergex-group-55, directories Richfield accommodation interface and tests.